Contents

- 1
- 2
- 3
- 4


```
% hw7
% Sherry Yang
```

1

a conditional density of f(x|y=y0, z= z0) using Bayes: f(x|y=y0,z=z0)dx = (integral from 0 to 1) C(4pi/3*xy0z0-V)^-2dx

```
% b
% invert the function so that it's f^-1 = C
%        if u < (4 * pi / 3 * x * y0 * z0 - V)^(-2):
%            return x


% c
% initialize random variables of x,y,z from 0 to 1
% create coniditonal density functions gives values of other variables by
% taking integral of one variables while fixating other variables.
% loop through k iterations
    % e.g. update x by sampling from conditional density while keeping y
    % and z fixed
% do the same for y and z , which creates a gradual convergence of samples
% to represent joint density f(x,y,z)
% function samples = gibbs_sampler(num_iterations, initial_values, V)
%     x = initial_values(1);
%     y = initial_values(2);
%     z = initial_values(3);
%
%     samples = zeros(num_iterations, 3);
%     samples(1, :) = [x, y, z];
%
%     for i = 2:num_iterations
%         x = sample_x_given_yz(y, z, V);
%
%         y = sample_y_given_xz(x, z, V);
%
%         z = sample_z_given_xy(x, y, V);
%
%         samples(i, :) = [x, y, z];
%     end
% end
```

2

```
% a
% see pdf

% b
% see pdf

% c
% lambda1/(2*(lambda1 + lambda3)) * 0 + 0 * (-2lambda1 - 2lambda2) +
% -(lambda1 - lambda3)/(4*(lambda1 + lambda3)) * (-2lambda2-2lambda3) )+
% 0*(-2lambda1 - 2lambda3)



% d final answer see pdf

% syms lambda1 lambda2 lambda3;
%
% mat = [-lambda2-2*lambda1, lambda2, lambda3, lambda3;
%           lambda2, -lambda2-2*lambda1, lambda3, lambda3;
%           lambda1, lambda1, -lambda2-2*lambda3, lambda2;
```

```
%           lambda1, lambda1, lambda2, -lambda2-2*lambda3];
%
% [V, D] = eig(mat);
%
% eigenvalues = diag(D);
%
% P0 = [1/4; 1/4; 1/4; 1/4];
% coefficients = V \ P0;
```

3

```
% a
% 1. Initialize the most recent common ancestor (MRCA) cell with no mutations and position (0,0,0).
% 2. Create an empty set of cells to store the population.
% 3. Loop through generations:
%     a. For each generation:
%         i. Create a new empty set for the next generation's cells.
%         ii. For each new cell in the current generation:
%             A. Randomly select two parent cells from the current population.
%             B. Calculate the time difference between generations.
%             C. Generate a Poisson-distributed number of mutations for the child cell.
%             D. Generate mutations uniformly distributed on [0,1].
%             E. Calculate the migration distance for the child cell using a Gaussian distribution.
%             F. Create a new child cell with mutations and position.
%             G. Assign the child cell's position by summing parent positions and migration.
%         iii. Update the current generation's population with the new generation's cells.
% 4. Return the set of cells containing information about mutations and spatial positions for each cell.


%b
% if hypothesis A were true where progeny of genetically more fit cells gradually migrate take over a tumor, such that
% physical position has little correlation to genetic ancestry (hypothesis A)
% there is no correlaation, C value will tell us no information about
% genetic and physcially proximity, and it will be a random distribution

% if hypothesis B were true where physically nearby cells tend to have similar genetics compared to physically distant cells
% when cells are close physically and geneticlaly, fi^2 has small value,
% but g will have smaller value, since union will be small compared to
% instersection, so C will be smaller.
% when cells are distant physiclaly and genetically, fi^2 will have large
% value, but g will have larger value, since union will be large compared
% to intersection
% C value will be larger when cells are distant physcially and genetically, therefore there will be strong correlation for C value



% c / d

cells_1 = coalescent_simulator(10,10,1,1000)
cells_2 = coalescent_simulator(10,1,10,1000)
cells_3 = coalescent_simulator(10,10,10,1000)


[mean_C1, std_C1] = coalescent_simulator_trials(10,10,1,1000,5)
[mean_C2, std_C2] = coalescent_simulator_trials(10,10,1,1000,5)
[mean_C3, std_C3] = coalescent_simulator_trials(10,10,1,1000,5)
 % by changing mutation rate/genetic distance parameter mu and physical
 % distance parameter sigma, C is close to 0 with very small/negligible
 % correlation. this means that correlation C does not vary changing either
 % parameter, which proves hypothesis A is right


% d
%
mat = [1,1,1,0,1
    1,1,0,0,0];

function cells = coalescent_simulator(k, mu, sigma, generations)
    MRCA = struct('mutations', [], 'location', [0, 0, 0]);
    cells = MRCA;

    f_squared = [];
    g = [];
```

```
    for gen = 1:generations
        next_gen = repmat(struct('mutations', [], 'location', [0, 0, 0]), k, 1);

        for i = 1:k
            if length(cells) < 2
                idx = randperm(length(cells));
                parent1 = cells(idx(1));
                parent2 = cells(idx(1));
            else
                idx = randperm(length(cells), 2);
                parent1 = cells(idx(1));
                parent2 = cells(idx(2));
            end

            time = gen;

            mutations = poissrnd(mu * time);

            child_mutations = rand(1, mutations);

            migration = normrnd(0, sigma * sqrt(time), 1, 3);

            child = struct('mutations', child_mutations, 'location', [0, 0, 0]);

            child.location = parent1.location + parent2.location + migration;

            next_gen(i) = child;


            f_squared = [f_squared, sum((child.location - parent1.location - parent2.location).^2)];

            g = [g, length(union(child.mutations, intersect(parent1.mutations, parent2.mutations)))];
        end

        next_gen_reshaped = reshape(next_gen, [], 1);
        cells = [cells; next_gen_reshaped];
    end


    n = length(f_squared);
    mean_f_squared = mean(f_squared);
    mean_g = mean(g);

    numerator = (1/n * sum(f_squared .* g)) - (1/n * sum(f_squared)) * (1/n * sum(g));
    denominator = sqrt((1/n * sum(f_squared)^2) - (1/n * sum(f_squared.^2))) * sqrt((1/n * sum(g.^2)) - (1/n * sum(g)^2));

    correlation_C = numerator / denominator;

    disp(['Correlation between genetic and physical distance (C): ', num2str(correlation_C)]);

    cells = correlation_C;

end

function [mean_C, std_C] = coalescent_simulator_trials(k, mu, sigma, generations, r)
    correlation_values = zeros(r, 1);

    for trial = 1:r
        disp(['Running trial ', num2str(trial)]);
        correlation_values(trial) = coalescent_simulator(k, mu, sigma, generations);
    end

    mean_C = mean(correlation_values);
    std_C = std(correlation_values);

    disp(['Mean of C across ', num2str(r), ' trials: ', num2str(mean_C)]);
    disp(['Standard deviation of C across ', num2str(r), ' trials: ', num2str(std_C)]);
end
```

```
Correlation between genetic and physical distance (C): 0-3.2624e-05i

cells_1 =

   0.0000e+00 - 3.2624e-05i
```

```
Correlation between genetic and physical distance (C): 0-3.3386e-05i

cells_2 =

   0.0000e+00 - 3.3386e-05i

Correlation between genetic and physical distance (C): 0-3.3431e-05i

cells_3 =

   0.0000e+00 - 3.3431e-05i

Running trial 1
Correlation between genetic and physical distance (C): 0-3.3035e-05i
Running trial 2
Correlation between genetic and physical distance (C): 0-3.3778e-05i
Running trial 3
Correlation between genetic and physical distance (C): 0-3.3615e-05i
Running trial 4
Correlation between genetic and physical distance (C): 0-3.2756e-05i
Running trial 5
Correlation between genetic and physical distance (C): 0-3.3761e-05i
Mean of C across 5 trials: 0-3.3389e-05i
Standard deviation of C across 5 trials: 4.6548e-07

mean_C1 =

   0.0000e+00 - 3.3389e-05i


std_C1 =

   4.6548e-07

Running trial 1
Correlation between genetic and physical distance (C): 0-3.3749e-05i
Running trial 2
Correlation between genetic and physical distance (C): 0-3.3036e-05i
Running trial 3
Correlation between genetic and physical distance (C): 0-3.4264e-05i
Running trial 4
Correlation between genetic and physical distance (C): 0-3.2809e-05i
Running trial 5
Correlation between genetic and physical distance (C): 0-3.3249e-05i
Mean of C across 5 trials: 0-3.3421e-05i
Standard deviation of C across 5 trials: 5.8508e-07

mean_C2 =

   0.0000e+00 - 3.3421e-05i


std_C2 =

   5.8508e-07

Running trial 1
Correlation between genetic and physical distance (C): 0-3.3369e-05i
Running trial 2
Correlation between genetic and physical distance (C): 0-3.2857e-05i
Running trial 3
Correlation between genetic and physical distance (C): 0-3.3342e-05i
Running trial 4
Correlation between genetic and physical distance (C): 0-3.2848e-05i
Running trial 5
Correlation between genetic and physical distance (C): 0-3.3449e-05i
Mean of C across 5 trials: 0-3.3173e-05i
Standard deviation of C across 5 trials: 2.9545e-07

mean_C3 =

   0.0000e+00 - 3.3173e-05i


std_C3 =
```

```
2.9545e-07
```

4

```
% a the max # node is the total number of edges, since the node cannot connect
% to itself

% b the transition probabliteis from V' to V'' depend on the moving piece:
% when removing a node from V' , for a given node v in V' the probablity of
% transitioning from V' to V'' by removing v depends on the Boltzmann
% weight. If the node s in the clique v', the transition probabliy withh be
% proportional to e ^ -(|V'|-1)/T, as the size of the clique -1 after removal

% when adding a node to V', if node v is not alreayd in V' but shraes edges
% with all nodes in v', the prob of transitioning from V' to V'' by adding
% v depends on the condition that it shares edges with all notes in V' the
% transiton probability will be proportional to e ^ -(|V'|-1)/T,since size
% of the clique + 1 after adding one more node

% c
% Initialization: start with an empty clique V' = {} or randomly chosen
% initial clique
% Sampling iteration
%   Iterate k times, where user supplied
    % in each iteration, generate a propsod clique(V'') by considering a random
    % node int he graph
        % if node already in the current clique V' , remove it friom the
        % proposed clique
        % if node is not in the current clique and is connected all nodes
        % in the current clique, add it to the propsoed clique

% calculate the accpetance probablity for transitioning from current
% clique(V') to the propsoed clique (V'') based on their sizes
    % compute energy of the current and propsoed cliques (basically size of
    % the clique)
    % caluculate boltzmann weights for btoh cliques
    % determine acceptance probility using ratio of boltzmann weights
    % (e^-|V''|/e^-|V'|)

% accet or reject the proposed clique
    % randomly decide whether to accpet the proposed clique based on its
    % acceptacne prob,. if accepted update the current clique to the
    % propsoed clique
% return final clique after k iteratiosn

% d

function main()
    n = 5;
    k = 10000;
    T = 1;
    r = 0.99;
    A = [
        1 1 1 0 1;
        1 1 0 0 0;
        1 0 1 1 1;
        0 0 1 1 0;
        1 0 1 0 1
    ];
    cliquesMetropolis = MetropolisSampler(A, k);

    fprintf('frequencies of observed cliques:\n');
    freqMetropolis = tabulate(cellfun(@(x) num2str(x), cliquesMetropolis, 'uniformoutput', false));
    for i = 1:size(freqMetropolis, 1)
        fprintf('%s: %d\n', freqMetropolis{i, 1}, freqMetropolis{i, 2})
    end

    finalClique = SimulatedAnnealingOptimizer(A, k, r, T);

    fprintf('\nfinal clique found by Optimizer:\n')
    disp(finalClique)
end
```

```
function cliques = MetropolisSampler(A, k)
    cliques = {};

    for step = 1:k
        currentClique = [];
        for i = 1:size(A, 1)
            if isempty(currentClique) || rand() < 0.5
                currentClique = [currentClique, i];
            elseif all(ismember(A(i, currentClique), 1))
                currentClique = [currentClique, i];
            end
        end
        cliques{step} = currentClique;
    end
end

function finalClique = SimulatedAnnealingOptimizer(A, k, r, T)
    finalClique = [];

    currentClique = [];
    for step = 1:k
        temperature = T * (r ^ step);

        proposedClique = GenerateProposedClique(A, currentClique);
        acceptanceProb = CalculateAcceptanceProbability(currentClique, proposedClique, temperature);
        if rand() < acceptanceProb
            currentClique = proposedClique;
        end
    end
    finalClique = currentClique;
end

function proposedClique = GenerateProposedClique(A, currentClique)
    proposedClique = currentClique;
    randomNode = randi(size(A, 1));

    if ismember(randomNode, proposedClique)
        isNeighbour = all(A(randomNode, proposedClique) == 1);
        if isNeighbour
            proposedClique = [proposedClique, randomNode];
        end
    end
end

function acceptanceProb = CalculateAcceptanceProbability(currentClique, proposedClique, temperature)
    energyCurrent = numel(currentClique);
    energyProposed = numel(proposedClique);
    weightCurrent = exp(-energyCurrent / temperature);
    weightProposed = exp(-energyProposed / temperature);
    acceptanceProb = min(1, weightProposed / weightCurrent);
end
```

2a) $$\frac{dp_{ij}(t)}{dt} = \sum_{k=1}^{n} p_{ik}(t)\lambda_{kj}$$

$$\frac{dp_A}{dt} = (-\lambda_2 - 2\lambda_1)\, p_A(t) + \lambda_2 p_T(t) + \lambda_3 (p_C(t) + p_G(t))$$

$$\frac{dp_T}{dt} = \lambda_2 p_A(t) + (-\lambda_2 - 2\lambda_1)\, p_T(t) + \lambda_3 (p_C(t) + p_G(t))$$

$$\frac{dp_C}{dt} = \lambda_1 (p_A(t) + p_T(t)) + (-\lambda_2 - 2\lambda_3)\, p_C(t) + \lambda_2 p_G(t)$$

$$\frac{dp_G}{dt} = \lambda_1 (p_A(t) + p_T(t)) + \lambda_2 p_C(t) + (-\lambda_2 - 2\lambda_3)\, p_G(t)$$

b)

$$\lambda = \begin{array}{c} \\ A \\ T \\ C \\ G \end{array} \begin{array}{c} \begin{array}{cccc} A & T & C & G \end{array} \\ \begin{bmatrix} -\lambda_2 - 2\lambda_1 & \lambda_2 & \lambda_3 & \lambda_3 \\ \lambda_2 & -\lambda_2 - 2\lambda_1 & \lambda_3 & \lambda_3 \\ \lambda_1 & \lambda_1 & -\lambda_2 - 2\lambda_3 & \lambda_2 \\ \lambda_1 & \lambda_1 & \lambda_2 & -\lambda_2 - 2\lambda_3 \end{bmatrix} \end{array}$$

1st is eigenvalue

$$\begin{bmatrix} -\cancel{\lambda_2\lambda_3} - 2\lambda_1\lambda_3 + \cancel{\lambda_2\lambda_3} + \cancel{2\lambda_1\lambda_3} \\ \lambda_2 + \lambda_2 + 2\lambda_1 \\ -\lambda_2 - 2\lambda_3 - \lambda_2 \\ 2\lambda_1 - \cancel{\lambda_2} + \cancel{\lambda_2} + 2\lambda_3 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ -2\lambda_1 - \lambda_2 \\ -2\lambda_2 - 2\lambda_3 \\ -2\lambda_1 - 2\lambda_3 \end{bmatrix}$$

d) $$\frac{\lambda_1}{2(\lambda_1 + \lambda_3)}$$